The code above prepares a global array to store the names of the required files; then it opens the Kernel module and plays a few tricks with method aliases; and finally, it loads the Rakefile. Focus on the middle part—the code dealing with Kernel. To understand what is going on there, look at this slightly simplified version of the original code:

```
module Kernel
  alias_method :require_without_record, :require

  def require(file)
    $requires << file if caller[1] =~ /rake2thor:/
    require_without_record file
  end
end
```

The *Open Class (14)* above does three things. First, it aliases the standard Kernel#require method to another name (require_without_record). Second, it *Monkey-patches (16)* require to store the names of files that are required by the Rakefile. (It does that by getting the stack of callers with the Kernel#callers method. If the second caller in the stack is rake2thor itself, this means that the Rakefile must be the first caller in the stack—the one that actually called require.) Finally, the new require falls back to the original require, now called require_without_record.

Compared to this simplified version, the original rake2thor code goes one step further: it also creates an alias for the new require called require_with_record. While this latest alias makes the methods more explicit, the important result is pretty much the same in both versions of the code: Kernel#require has changed, and the new require is "wrapped around" the old require. That's why this trick is called an *Around Alias*.

Spell: Around Alias

You can write an Around Alias in three simple steps:

1. You alias a method.
2. You redefine it.
3. You call the old method from the new method.

One downside of Around Aliases is that they pollute your classes with one additional method name. You can fix this small problem somehow by making the old version of the method private after you alias it. (In Ruby it's the method's name, not the method itself, that is either public or private.)

Another potential problem of Around Aliases has to do with loading. You should never load an Around Alias twice, unless you want to end up with an exception when you call the method. Can you see why?